

并行程序设计 Lab2 SIMD 编程实验报告

口令猜测中的 MD5 SIMD 加速

姓名：刘蔚霖 学号：2412727

2026 年 5 月 9 日

目录

1	实验环境与评价指标	2
1.1	实验环境	2
1.2	实验流程	2
1.3	评价指标	3
2	算法迭代设计：为什么能加速（基础要求）	3
2.1	数据级并行思路	3
2.2	关键代码	4
2.3	降低额外开销的实现细节	5
2.3.1	减少字符串复制	6
2.3.2	将 SIMD 轮函数宏展开	7
2.3.3	使用常量移位指令	7
2.3.4	短口令单块 fast path	7
2.4	实现改动小结	8
3	正确性验证（基础要求）	9
4	编译选项影响分析（进阶要求）	10
5	达标加速比（进阶要求）	12
6	perf 基础分析（基础要求）	13
7	perf 进阶分析（进阶要求）	14
7.1	热点函数	14
7.2	编译选项与汇编	15

摘要

本实验选择口令猜测任务中的 MD5 哈希阶段作为 SIMD 加速对象，在 ARM OpenEuler 服务器上使用 NEON Intrinsics 实现 4 路并行 MD5。本文先说明 4 路 NEON 并行的实现方式和优化细节，再验证串行 MD5 与 SIMD MD5 的正确性，随后比较 -O0/-O1/-O2 下的加速比，并使用 `perf stat`、`perf report` 和反汇编分析性能瓶颈。当前 SIMD 版本在 -O1/-O2 下的 Hash 阶段加速比分别为 2.61x 和 2.74x，已达到 2.0x 以上要求。

代码仓库: https://github.com/Aokiuuz/nku_parallel1.git

1 实验环境与评价指标

1.1 实验环境

实验平台为 ARM OpenEuler 服务器，目标指令集为 AArch64，SIMD 扩展为 ARM NEON。代码使用 C++ 实现，串行版本调用标量 MD5Hash，并行版本调用 MD5Hash_simdversion。性能采集工具包括 `perf stat`、`perf record`、`perf report` 和 `objdump`。为避免单次运行波动，所有 `perf stat` 和运行时间均采用 3 次重复运行的平均值。

表 1: 实验环境

项目	配置
运行平台	ARM OpenEuler 服务器
处理器架构	AArch64
SIMD 扩展	ARM NEON, 128-bit 向量寄存器
编程语言	C++, NEON Intrinsics
编译选项	-O0、-O1、-O2
性能工具	<code>perf stat</code> 、 <code>perf record/report</code> 、 <code>objdump</code>
测试数据	口令训练集与候选口令生成结果，候选规模约 10^7

1.2 实验流程

实验流程分为五步。第一，设计算法，确定将 4 个独立候选口令映射到 NEON 的 4 个 32-bit lane 中并行执行 MD5。第二，分别编译串行版本和 SIMD 版本，并在 -O0/-O1/-O2 三种优化等级下生成独立可执行文件。第三，运行正确性测试，比较 SIMD 每个 lane 的输出与串行 MD5 输出是否一致。第四，记录 Hash 阶段运行时间，计算 Hash 加速比。第五，使用 `perf` 统计硬件计数器，并对 SIMD 版本进行采样和反汇编分析。

Listing 1: 实验采集流程

```

1 # 1. 设计算法：4 路 NEON SIMD 并行 MD5
2
3 # 2. 编译 00/01/02 版本
4 g++ -std=c++11 -O0 *.cpp -o main_00
5 g++ -std=c++11 -O1 *.cpp -o main_01
6 g++ -std=c++11 -O2 *.cpp -o main_02
7
8 # 3. 运行正确性测试并记录输出
9 ./main_02 > data/perf_results/02_simd_run.out
10
11 # 4. 采集 Hash time 并计算 Hash 加速比
12
13 # 5. 采集 perf stat、热点和反汇编
14 perf stat -r 3 -e cycles,instructions,branch-misses,cache-references,cache-
    misses ./main_02
15 perf record -g ./main_02
16 perf report --stdio > data/perf_results/02_simd_report.txt
17 objdump -d ./main_02 > data/perf_results/02_simd_main.asm

```

1.3 评价指标

本文主要使用 Hash 加速比。Hash 加速比只比较 MD5 计算阶段，能够直接反映 SIMD 化对哈希函数本身的优化效果：

$$S_{\text{hash}} = \frac{T_{\text{hash, serial}}}{T_{\text{hash, SIMD}}}.$$

训练阶段 Train time 和候选生成阶段 Guess time 不参与加速比计算，因为训练和候选生成不是本次 SIMD 优化目标；但在整程序 perf 分析中仍保留这些阶段，用于解释完整程序热点分布。

2 算法迭代设计：为什么能加速（基础要求）

2.1 数据级并行思路

口令猜测程序由 PCFG 模型训练、候选口令生成和 MD5 哈希三个主要阶段组成。候选生成阶段维护优先队列并产生字符串，哈希阶段对大量独立候选口令计算 MD5。由于不同口令之间的 MD5 计算互不依赖，且 MD5 的 64 轮计算主要由 32 位整数加法、位运算和循环左移组成，适合使用 SIMD 指令进行数据级并行。

MD5 对每个 512-bit 消息块执行 64 轮迭代，内部维护 4 个 32-bit 状态字 A, B, C, D 。每一轮的输入包括当前状态、一个 message word、一个常量和一个循环左移位数。对单个口令而言，这 64 轮之间存在严格的数据依赖，不能简单把同一个口令的 64 轮拆开并行；但不同口令之间完全独立，因此可以把“口令维度”作为并行维度。也就是说，SIMD 并不是并行化同一个 MD5 的轮次，而是同时执行 4 个 MD5。

本实验的核心思想是将 4 个候选口令分别放入 NEON 向量的 4 个 lane 中。MD5 状态变量 A, B, C, D 从标量 `bit32` 改为 `uint32x4_t`，每个 lane 对应一个口令。这样一次 SIMD 调用可同步处理 4 个口令：

$$\begin{aligned} A &= (A_0, A_1, A_2, A_3), \\ B &= (B_0, B_1, B_2, B_3), \\ C &= (C_0, C_1, C_2, C_3), \\ D &= (D_0, D_1, D_2, D_3). \end{aligned}$$

对于 MD5 中的 F/G/H/I、FF/GG/HH/II 轮函数，标量位运算被替换为 NEON 的 `vandq_u32`、`vorrq_u32`、`veorq_u32`、`vaddq_u32` 等向量操作。每轮的数学含义不变，只是同时作用于 4 个 lane。

表 2: 标量 MD5 到 NEON SIMD 的映射

标量操作	NEON 操作	作用
$x + y$	<code>vaddq_u32(x, y)</code>	4 个 lane 同时做 32-bit 加法
$x \& y$	<code>vandq_u32(x, y)</code>	4 个 lane 同时做按位与
$x y$	<code>vorrq_u32(x, y)</code>	4 个 lane 同时做按位或
$x \wedge y$	<code>veorq_u32(x, y)</code>	4 个 lane 同时做按位异或
$\sim x$	<code>vmvnq_u32(x)</code>	4 个 lane 同时取反
循环左移	<code>vshlq_n_u32/vshrq_n_u32</code>	常量移位后合并

因此，理论上一次 SIMD 调用能覆盖 4 个候选口令的 MD5 计算。如果忽略装载、padding、尾部处理和寄存器调度开销，Hash 阶段的理想上限接近 4x。实际测得 -01/-02 下为 2.61x 和 2.74x，说明主要收益来自 4 路数据并行，但仍受到消息预处理、内存访问和编译器生成代码质量的限制。

2.2 关键代码

SIMD 版本的接口如下：

Listing 2: SIMD MD5 接口

```
1 void MD5Hash(const string &input, bit32 *state);
2 void MD5Hash_simdversion(const string input[4], uint32x4_t state[4]);
```

SIMD 版本的基本轮函数将标量宏改写为 NEON 运算。例如：

Listing 3: NEON 轮函数

```
1 #define F_SIMD(x, y, z) \
2     (vorrq_u32(vandq_u32((x), (y)), vandq_u32(vmvnq_u32((x)), (z))))
3
4 #define ROTATELEFT_SIMD(num, n) \
5     (vorrq_u32(vshlq_n_u32((num), (n)), vshrq_n_u32((num), 32 - (n))))
6
7 #define FF_SIMD(a, b, c, d, x, s, ac) \
8     do { \
9         (a) = vaddq_u32((a), \
10             vaddq_u32(F_SIMD((b), (c), (d)), \
11                 vaddq_u32((x), vdupq_n_u32(ac)))); \
12         (a) = ROTATELEFT_SIMD((a), (s)); \
13         (a) = vaddq_u32((a), (b)); \
14     } while (0)
```

在主程序中，每当候选口令积累到一定规模后，将其按 4 个一组送入 SIMD MD5。若剩余口令不足 4 个，则继续用串行 MD5 处理，避免越界。

Listing 4: main.cpp 中的批量哈希

```
1 uint32x4_t state[4];
2 size_t i = 0;
3 for (; i + 3 < guesses.size(); i += 4)
4 {
5     MD5Hash_simdversion(&guesses[i], state);
6 }
7
8 bit32 tail_state[4];
9 for (; i < guesses.size(); i += 1)
10 {
11     MD5Hash(guesses[i], tail_state);
12 }
```

2.3 降低额外开销的实现细节

guess_2（具体代码详见 github 库）是从标量 MD5 到 SIMD MD5 的初改版，关键改动是先把并行维度确定为“口令维度”。标量版本一次只维护一组 A, B, C, D 状态，guess_2 则把状态改为 uint32x4_t，让 4 个 lane 分别保存 4 个候选口令的状态；同

时把 MD5 的 F/G/H/I 逻辑函数和 FF/GG/HH/II 轮函数改写为 NEON Intrinsics，用 `vaddq_u32`、`vandq_u32`、`vorrq_u32`、`veorq_u32` 等向量指令同步完成 4 份 MD5 计算。主程序也相应改为每 4 个候选口令组成一批调用 `MD5Hash_simdversion`，尾部不足 4 个的口令仍由串行 `MD5Hash` 处理。这个版本已经体现了 SIMD 的核心收益：同一条向量指令同时服务 4 个互不依赖的口令。

从 `guess_2` 初改版到当前版本，算法迭代的核心是减少 SIMD 计算前后的额外开销。初改版仍保留了较多标量程序的组织方式，例如每 4 个口令先构造临时 `string[4]`，每个 lane 分别调用 `StringProcess` 做动态 padding，SIMD 轮函数以普通函数形式调用，循环左移也使用变量移位实现。这些操作不属于 MD5 核心轮函数，却会在大量候选口令上反复发生，抵消一部分 4 路并行收益。当前版本在此基础上继续迭代：直接传入连续字符串地址，减少口令批量打包复制；将短口令单块 padding 改为栈上构造 message word，减少 `new/delete` 和内存初始化；把轮函数展开并使用常量移位指令，降低函数调用与移位控制开销。因此，当前版本的加速来自两部分叠加：一是 4 个独立 MD5 的数据级并行，二是减少 SIMD 外围的字符串、padding 和函数调用成本。

2.3.1 减少字符串复制

原始批量哈希写法中每 4 个口令会构造一次 `pw_batch`，这会触发 4 次字符串拷贝。当前写法将 SIMD 函数接口改为接收连续字符串地址，主程序直接传入候选口令数组中的当前位置：

Listing 5: 避免 string 批量复制

```
1 void MD5Hash(const string &input, bit32 *state);
2 void MD5Hash_simdversion(const string input[4], uint32x4_t state[4]);
3
4 static void HashGuessesSimd(const vector<string> &guesses)
5 {
6     uint32x4_t state[4];
7     size_t i = 0;
8     for (; i + 3 < guesses.size(); i += 4)
9     {
10         MD5Hash_simdversion(&guesses[i], state);
11     }
12
13     bit32 tail_state[4];
14     for (; i < guesses.size(); i += 1)
15     {
16         MD5Hash(guesses[i], tail_state);
17     }
18 }
```

2.3.2 将 SIMD 轮函数宏展开

函数式写法中 `F_simd`、`FF_simd` 等为普通函数。在 `-O0` 下编译器通常不会内联这些函数，MD5 的 64 轮计算会产生大量函数调用开销。当前写法将其改为宏展开，使 `-O0` 下也能避免该开销：

Listing 6: 宏展开轮函数

```
1 #define F_SIMD(x, y, z) \
2     (vorrq_u32(vandq_u32((x), (y)), vandq_u32(vmvnq_u32((x)), (z))))
3
4 #define ROTATELEFT_SIMD(num, n) \
5     (vorrq_u32(vshlq_n_u32((num), (n)), vshrq_n_u32((num), 32 - (n))))
6
7 #define FF_SIMD(a, b, c, d, x, s, ac) \
8     do { \
9         (a) = vaddq_u32((a), \
10             vaddq_u32(F_SIMD((b), (c), (d)), \
11                 vaddq_u32((x), vdupq_n_u32(ac)))); \
12         (a) = ROTATELEFT_SIMD((a), (s)); \
13         (a) = vaddq_u32((a), (b)); \
14     } while (0)
```

2.3.3 使用常量移位指令

MD5 每轮循环左移的位数是编译期常量。当前写法将变量移位改为 `vshlq_n_u32` 和 `vshrq_n_u32`，减少额外的移位控制向量构造，并让编译器更容易生成直接的 NEON 常量移位指令。

2.3.4 短口令单块 fast path

Rockyou 数据集的大量口令长度不超过 55 字节。此时 MD5 padding 后只需要一个 512-bit block。原始写法仍然对每个 lane 调用 `StringProcess`，产生 `new[]`、`memcpy`、`memset` 和 `delete[]` 开销。当前写法对长度不超过 55 的 4 路口令直接在栈上构造 16 个 message word：

Listing 7: 短口令 fast path

```
1 static bool BuildSingleBlockWordsSimd(const string input[4], uint32x4_t x[16])
2 {
```

```

3      uint32_t lanes[16][4] = {};
4
5      for (int lane = 0; lane < 4; lane += 1)
6      {
7          const size_t length = input[lane].size();
8          if (length > 55)
9          {
10             return false;
11         }
12
13         const unsigned char *data =
14             reinterpret_cast<const unsigned char *>(input[lane].data());
15         for (size_t i = 0; i < length; i += 1)
16         {
17             lanes[i >> 2][lane] |=
18                 static_cast<uint32_t>(data[i]) << ((i & 3) * 8);
19         }
20
21         lanes[length >> 2][lane] |= 0x80u << ((length & 3) * 8);
22         const uint64_t bitLength = static_cast<uint64_t>(length) * 8;
23         lanes[14][lane] = static_cast<uint32_t>(bitLength);
24         lanes[15][lane] = static_cast<uint32_t>(bitLength >> 32);
25     }
26
27     for (int word = 0; word < 16; word += 1)
28     {
29         x[word] = vld1q_u32(lanes[word]);
30     }
31     return true;
32 }

```

若遇到长指令或 4 个 lane 的 padding 后块数不一致，则当前实现会退回慢路径或串行打包结果，以保证正确性。

2.4 实现改动小结

本次 SIMD 版本并不是只把标量变量替换为向量变量，还针对指令猜测程序的调用方式做了多处开销控制。主要改动如表 3 所示。

表 3: SIMD 实现与优化点

优化点	解决的问题	对性能的影响
4 路 <code>uint32x4_t</code> 状态	单次只处理一个口令	将 4 个独立 MD5 合并执行
宏展开轮函数	-00 下函数调用频繁	减少 64 轮计算中的调用边界
连续字符串指针传入	批量构造 <code>string[4]</code> 产生复制	降低候选口令打包开销
常量移位 <code>Intrinsics</code>	变量移位需要额外控制向量	便于生成直接移位指令
单块 fast path	短口令仍走动态 padding	减少 <code>new/delete</code> 与内存初始化
尾部串行处理	候选数量不是 4 的倍数	保证边界正确性

这些改动共同决定了最终加速效果。4 路状态变量提供理论并行度，宏展开和常量移位降低计算核心开销，连续字符串传入与 fast path 则减少 SIMD 计算外部的准备成本。若只做向量化而不处理这些外围开销，实际加速比会被字符串复制和消息填充抵消。

3 正确性验证（基础要求）

程序启动时首先使用常见口令及其标准 MD5 值验证串行 MD5Hash，测试样例包括 123456、password、qwerty 等。当前 SIMD 版本还额外验证 MD5Hash_simdversion 的 4 个 lane 输出：每 4 个口令同时送入 SIMD 函数，再将每个 lane 的结果与对应口令的串行 MD5 结果比较。

表 4: 部分 MD5 正确性测试样例

输入口令	标准 MD5 值
123456	e10adc3949ba59abbe56e057f20f883e
password	5f4dcc3b5aa765d61d8327deb882cf99
qwerty	d8578edf8458ce06fbc5bb76a58c5ca4
abc	900150983cd24fb0d6963f7d28e17f72

实验输出中出现如下信息：

Listing 8: MD5 正确性验证输出

```

1 Testing MD5Hash correctness...
2 MD5Hash test passed!
3 Testing MD5Hash_simdversion correctness...
4 MD5Hash_simdversion test passed!
```

此外，各次运行训练阶段均输出 total pts18966，说明训练集读取和 PCFG 模型构建正常。因此后续性能数据是在串行 MD5、SIMD MD5 和训练流程均正确的前提下获得的。

4 编译选项影响分析（进阶要求）

本节使用 `data/perf_results` 目录下三次重复运行的均值作为实验结果。根据 GCC 优化选项文档，`-O0` 主要用于降低编译时间并保留调试行为，大多数优化 pass 不运行；`-O1` 开始优化代码大小和执行时间；`-O2` 会打开 `-O1` 的优化并继续启用更多优化 pass，例如函数内联、调度和循环相关优化。¹ 因此同一份 C++/NEON Intrinsics 代码在不同编译选项下会生成明显不同的机器码。

对本实验而言，编译选项的影响尤其明显。MD5 轮函数由大量短小的位运算和加法组成，如果编译器不能有效内联和分配寄存器，程序会产生更多临时变量保存、栈访问和函数调用边界。`-O1` 开始减少这类冗余开销，因此 SIMD 版本开始体现 4 路并行收益。`-O2` 进一步改善指令调度和公共表达式处理，使 NEON Intrinsics 更接近手写汇编的执行形态。

表 5: 不同编译优化等级下的 Hash time 与加速比

选项	串行 Hash/s	SIMD Hash/s	Hash 加速比
-O0	9.61910	10.30533	0.9334
-O1	3.16619	1.21392	2.6082
-O2	3.22361	1.17647	2.7401

图 1 进一步展示了不同优化等级下 Hash time 的变化。可以看到，`-O0` 下 SIMD 版本并没有获得收益；而从 `-O1` 开始，SIMD 版本的 Hash time 明显低于串行版本。`-O2` 下串行 Hash time 为 3.22361s，SIMD Hash time 为 1.17647s，说明编译器优化后 NEON 向量化路径成为主要有效路径。

¹<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

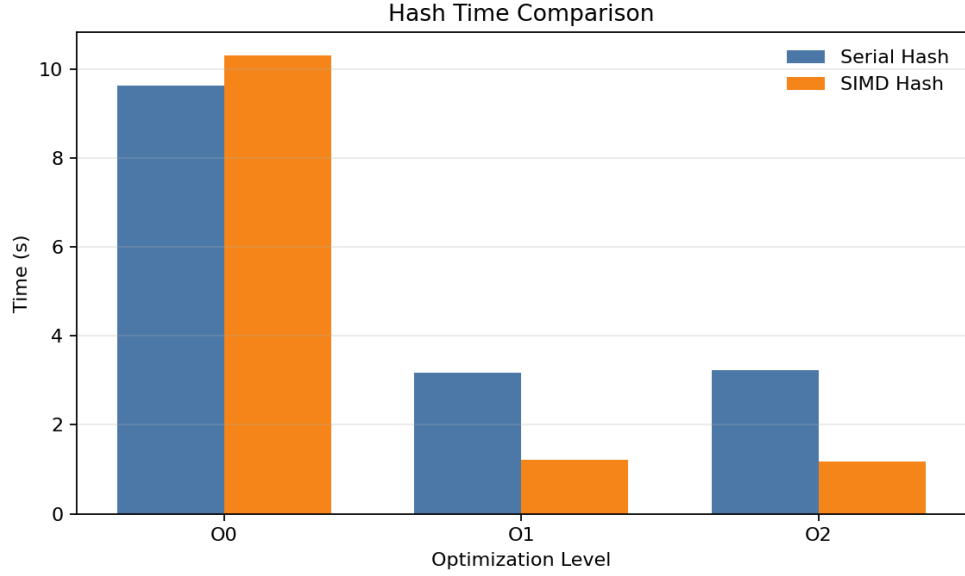


图 1: 不同编译优化等级下的 Hash time 对比

其中 Hash 加速比定义为串行 Hash time 除以 SIMD Hash time。本文所有运行时间对比表均只比较 Hash 阶段，不把候选生成和训练时间计入加速比。

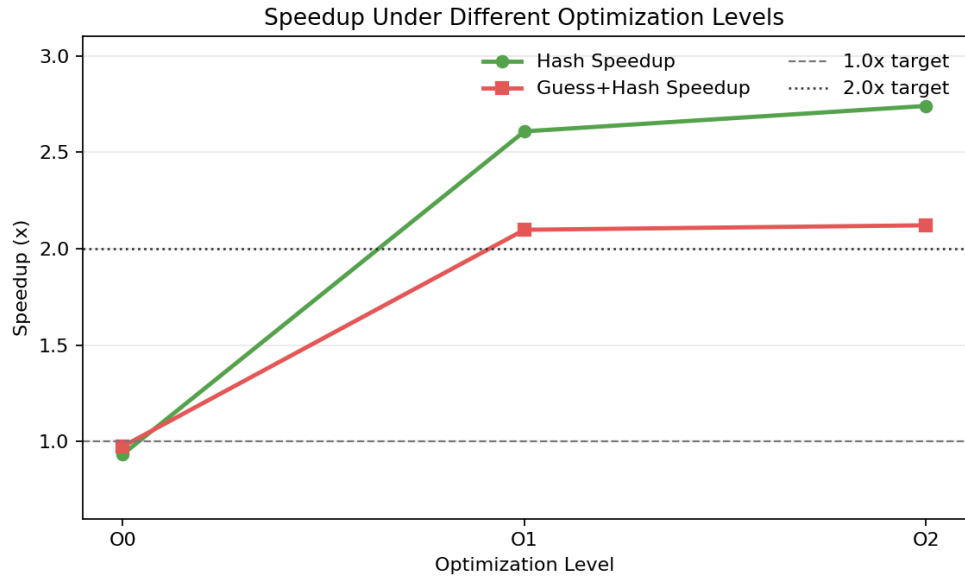


图 2: 不同编译优化等级下的加速比

图 2 中，-O1 和 -O2 的 Hash 加速比均超过 2.0x。-O0 的 Hash 加速比低于 1.0x，说明关闭优化后 SIMD Intrinsics 并不会自动带来收益，必须依赖编译器完成较好的指令选择、寄存器分配和冗余代码消除。

表 6: 不同编译优化等级下的 Hash time

选项	串行 Hash/s	SIMD Hash/s
-O0	9.61910	10.30533
-O1	3.16619	1.21392
-O2	3.22361	1.17647

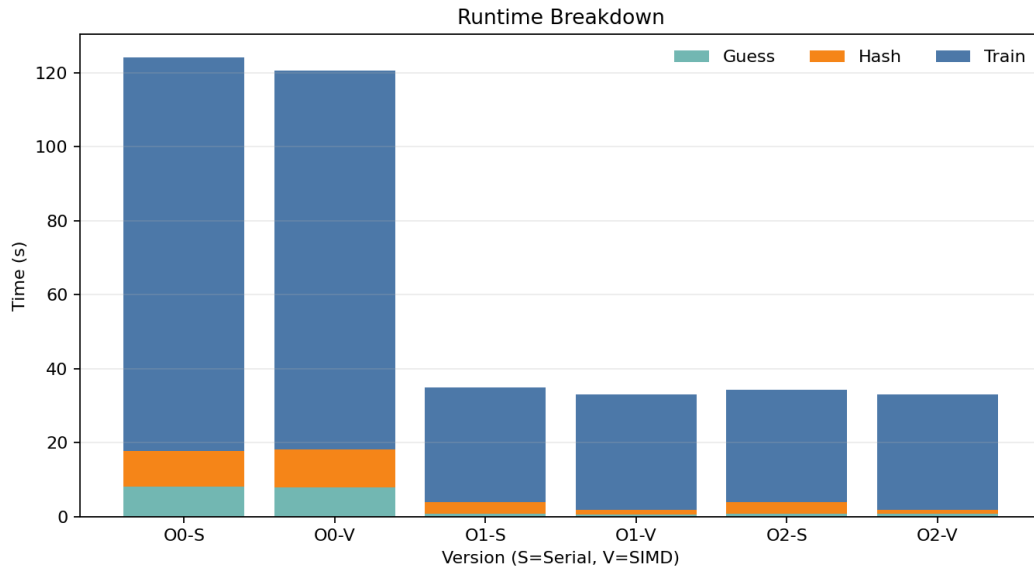


图 3: Guess、Hash、Train 三阶段运行时间分解

从图 3 可以看出：

- -O1 下 SIMD 的 Hash 加速比为 2.6082，超过 2.0x。
- -O2 下 SIMD 的 Hash 加速比为 2.7401，达到 2.0x 以上。
- -O0 下 SIMD 的 Hash time 为 10.30533s，慢于串行版本的 9.61910s，说明在关闭编译优化后，NEON Intrinsics 展开后的寄存器调度、临时变量保存和函数边界开销仍然较重。后续仍将把 -O0 纳入 perf 和汇编对照，用于解释未达标原因；只是本阶段不继续投入代码优化来提高 -O0 加速比。

5 达标加速比（进阶要求）

当前实验生成候选口令数量约为 10^7 。为保持数据口径一致，本文统一采用 data/perf_results 中三次重复运行的均值；旧历史数据不再作为实验结论使用。-O2 下的 Hash time 如下：

Hash 阶段加速比为：

表 7: 当前 advanced 版本 -02 Hash time

版本	Hash time / s
串行对照	3.22361
SIMD 版本	1.17647

$$S_{\text{hash}} = \frac{3.22361}{1.17647} = 2.7401.$$

表 8: 进阶目标完成情况

编译选项	目标加速比	实测 Hash 加速比	是否达标
-00	> 1.0	0.9334	暂未达标
-01	≥ 2.0	2.6082	达标
-02	≥ 2.0	2.7401	达标

6 perf 基础分析（基础要求）

本节记录 `perf stat` 的基础性能计数。根据 Linux perf 文档，`perf stat` 会运行目标命令并汇总硬件/软件性能计数器，因此适合比较串行版本和 SIMD 版本的 cycles、instructions 和 IPC。² 实验使用如下命令采集性能数据，结果保存在 `data/perf_results/*_perf.txt` 中：

Listing 9: perf stat 采集命令

```
1 perf stat -r 3 -e cycles,instructions,branches,branch-misses,cache-references,
  cache-misses ./main_02
```

采集 -00/-01 时将二进制名替换为 `main_00/main_01`。服务器上 `branches:u` 事件返回 `<not supported>`，因此表中不列分支总数；`branch-misses`、`cache` 事件、`cycles` 和 `instructions` 均正常记录。

从基础计数可以得到三个结论。第一，编译优化对总指令数影响非常明显：串行版本从 -00 的 3.560×10^{11} 条指令降到 -02 的 5.578×10^{10} 条，SIMD 版本也从 3.365×10^{11} 条降到 4.313×10^{10} 条。第二，在 -01/-02 下，SIMD 版本的总指令数均低于串行版本，说明 4 路并行确实减少了每个候选口令平均承担的 MD5 计算指令。第三，整程序 elapsed time 的改善小于 Hash time 改善，因为 `perf stat` 统计的是完整程序，训练和排序阶段仍占用了大量时间。

²<https://man7.org/linux/man-pages/man1/perf-stat.1.html>

表 9: perf stat 统计结果（三次运行均值）

版本	cycles	instructions	IPC	branch-misses	cache miss 率	elapsed/s
串行 -00	3.457e11	3.560e11	1.03	1.022e9	0.609%	135.16
SIMD -00	3.360e11	3.365e11	1.00	9.827e8	0.605%	131.444
串行 -01	1.007e11	6.029e10	0.60	6.526e8	4.454%	39.751
SIMD -01	9.542e10	4.717e10	0.49	6.413e8	5.109%	37.585
串行 -02	9.860e10	5.578e10	0.57	6.538e8	5.539%	38.845
SIMD -02	9.564e10	4.313e10	0.45	6.524e8	6.776%	37.772

进一步比较串行和 SIMD 的指令数可知，-01 下 SIMD 版本比串行版本少约 21.8% 的指令，-02 下少约 22.7% 的指令。这个比例小于 Hash 阶段 2.6x-2.7x 的加速比，原因是 perf 统计覆盖完整程序，训练、排序、候选生成等非 SIMD 部分仍会贡献大量指令。换言之，perf stat 反映的是整程序视角，而 Hash time 反映的是被优化函数所在阶段的局部视角，两者并不矛盾。

7 perf 进阶分析（进阶要求）

在基础计数之外，本实验使用 perf record 采集 SIMD 版本调用栈样本，并用 perf report 查看热点函数；随后用 objdump 检查编译器是否将 C++/NEON Intrinsics 转换成预期的 NEON 汇编。³ 由于前文已经说明串行版本难以获得稳定、集中的 MD5 热点，本节热点分析只针对 SIMD 版本展开；串行版本保留 perf stat 作为对照。

Listing 10: 热点和汇编分析命令

```

1 perf record -g ./main
2 perf report --stdio > perf_report.txt
3 objdump -d main > main.asm

```

7.1 热点函数

perf report 输出均位于 data/perf_results 目录，对应文件分别为 00_simd_report.txt、01_simd_report.txt 和 02_simd_report.txt。主要热点如下：

表 10: SIMD 版本 perf report 主要热点

版本	model::train	model::order	segment::order	MD5Hash_simdversion
-00	54.86%	23.06%	15.82%	7.71%
-01	53.46%	20.89%	20.73%	3.66%
-02	54.34%	20.54%	19.43%	2.75%

³<https://man7.org/linux/man-pages/man1/perf-record.1.html>

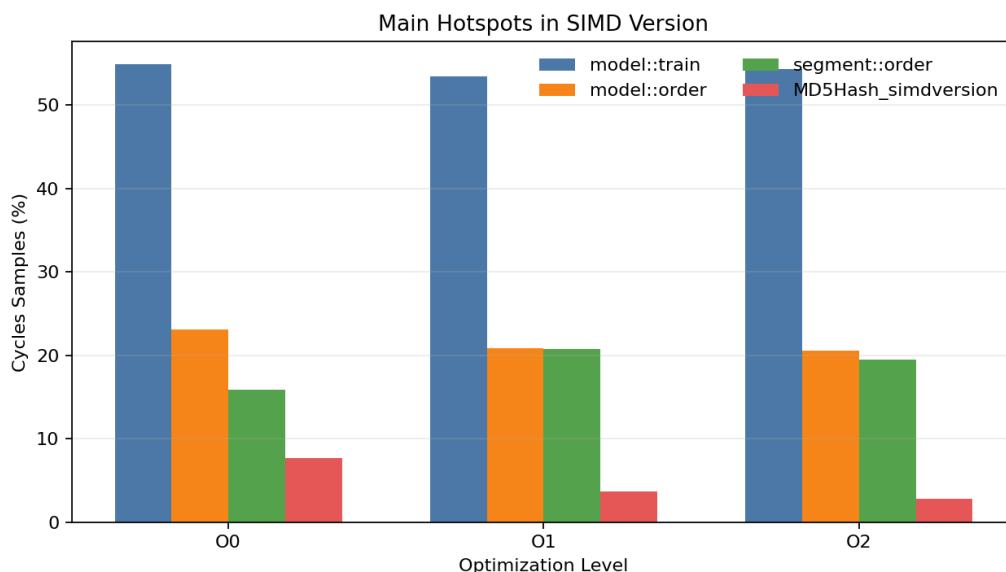


图 4: SIMD 版本主要热点占比

这个结果说明整程序热点主要集中在 PCFG 训练、模式统计和排序上，而不是 MD5 本身。其原因是 `perf record` 采样的是完整程序，训练阶段需要读取、解析和统计大量口令，`model::order/segment::order` 内部又包含大量排序和字符串比较。因此即使 Hash 阶段已经通过 SIMD 获得 2x 以上加速，在整程序热点图中 `MD5Hash_simdversion` 仍只占较小比例。

从图 4 可以看到，`model::train` 在三个优化等级下都稳定占据约 53%–55% 的采样比例，说明训练逻辑是整程序最主要的固定开销。`MD5Hash_simdversion` 的占比从 -O0 的 7.71% 降到 -O2 的 2.75%，一方面是因为编译优化显著减少了 MD5 相关指令，另一方面也说明 Hash 阶段优化后在完整程序中的权重下降。此时继续只优化 MD5 的边际收益会逐渐变小。

7.2 编译选项与汇编

反汇编结果均位于 `data/perf_results` 目录，对应文件分别为 `O0_simd_main.asm`、`O1_simd_main.asm` 和 `O2_simd_main.asm`。三个文件均可定位到 `MD5Hash_simdversion` 符号，说明 SIMD MD5 没有被错误替换为纯串行实现。-O1/-O2 下反汇编中可以观察到以 NEON 向量寄存器为操作对象的加法、异或、或、移位和访存指令，对应 C++ 代码中的 `vaddq_u32`、`veorq_u32`、`vorrq_u32`、`vshlq_n_u32/vshrq_n_u32` 以及 `vld1q_u32` 等 Intrinsics。

Listing 11: -O2 下 `MD5Hash_simdversion` 的部分 NEON 汇编

```

1 411114: add v0.4s, v7.4s, v0.4s
2 411124: shl v3.4s, v0.4s, #7
3 411128: ushr v0.4s, v0.4s, #25

```

```

4 411138: orr v0.16b, v3.16b, v0.16b
5 411168: eor v1.16b, v1.16b, v10.16b
6 411190: shl v2.4s, v1.4s, #12
7 411194: ushr v1.4s, v1.4s, #20
8 4111a4: orr v2.16b, v2.16b, v1.16b

```

其中 `add v?.4s` 表示 4 个 32-bit lane 同时加法，`eor/orr` 对应 MD5 轮函数中的按位逻辑运算，`shl + ushr + orr` 组合对应循环左移。这说明编译器确实将 NEON Intrinsics 降低为向量汇编指令，而不是展开为 4 次独立的标量 MD5。

从性能计数和汇编规模看，`-O0` 下 SIMD 版本仍有较重开销。其反汇编文件约 1.61 MB，而 `-O1/-O2` 分别约 0.98 MB 和 0.92 MB；同时 `-O0` SIMD 的总指令数为 3.365×10^{11} ，远高于 `-O2` SIMD 的 4.313×10^{10} 。这说明 `-O0` 没有充分进行函数内联、寄存器分配和冗余代码消除，导致 SIMD 计算节省下来的数据并行收益被额外指令和访存开销抵消。因此 `-O0` Hash 加速比只有 0.9334，未达到 1.0x。

`-O1/-O2` 下，编译器能更好地优化 NEON Intrinsics 生成的机器码。与串行版本相比，`-O1` SIMD 指令数从 6.029×10^{10} 降至 4.717×10^{10} ，`-O2` SIMD 指令数从 5.578×10^{10} 降至 4.313×10^{10} ；配合 4 路 lane 并行，Hash 阶段分别达到 2.6082x 和 2.7401x。短口令 fast path 还减少了 `StringProcess`、动态分配和内存初始化开销，使热点进一步集中到真正的 MD5 轮函数计算上。